Звіт про архітектуру

1. Архітектурний підхід: Clean Architecture

Проєкт побудовано з використанням принципів "Чистої архітектури" (Clean Architecture). Це рішення забезпечує чіткий поділ відповідальності між компонентами системи, гнучкість у підтримці та полегшує майбутнє масштабування.

Solution розділено на чотири незалежні шари (Class Libraries), де залежності спрямовані виключно вглиб – до ядра:

- Domain: Містить базові бізнес-сутності та інтерфейси. Не має залежностей від інших проєктів.
- Application: Інкапсулює бізнес-логіку (Use Cases). Залежить лише від шару Domain.
- Infrastructure: Реалізує роботу з базою даних, зовнішніми сервісами та інфраструктурою. Залежить від Application (реалізує його інтерфейси).
- Api (Presentation): Головний застосунок, налаштований за патерном MVC. Відповідає за прийом HTTP-запитів, DI-контейнер (Dependency Injection) та маршрутизацію (Controllers). Залежить від Application та Infrastructure.

Перевага вибору: Такий підхід робить бізнес-логіку незалежною від фреймворків, баз даних чи UI, що дозволяє легко тестувати ядро застосунку (Unit-тести) та змінювати технологічний стек інфраструктури без переписування основної логіки.

2. Робота з даними: PostgreSQL + Entity Framework Core

Для зберігання даних обрано реляційну систему керування базами даних (СКБД) PostgreSQL у поєднанні з ORM Entity Framework Core.

- PostgreSQL обрано через її надійність, відповідність стандартам SQL, потужний функціонал та відкритий вихідний код. Вона чудово підходить для складних реляційних структур.

- Entity Framework Core (з провайдером Npgsql) використовується як стандартний і найбільш інтегрований інструмент для екосистеми .NET.

Інфраструктуру налаштовано з використанням підходу Code-First. У проєкті Infrastructure створено ApplicationDbContext, а генерація та застосування міграцій дозволяють версіонувати схему БД і розгортати її автоматизовано.

3. Інструменти контролю якості: Roslynator та StyleCop

Для забезпечення єдиного стилю написання коду всією командою, превентивного виявлення помилок та автоматизації перевірки форматування інтегровано два потужні статичні аналізатори коду – Roslynator.Analyzers та StyleCop.Analyzers.

Ці інструменти доповнюють один одного:

- Roslynator фокусується на виявленні логічних помилок, «запахів коду» (code smells) та пропонує оптимізації продуктивності.
- StyleCop жорстко контролює стилістику C#-коду: дотримання правильних відступів, конвенцій найменування змінних, правильний порядок елементів у класах та обов'язковість документації.

Реалізація в проєкті:

- Обидва пакети встановлені в усі проєкти Solution.
- У корінь репозиторію додано конфігураційні файли: .editorconfig (як єдине джерело істини для базового форматування) та stylecop.json (для тонкого налаштування специфічних правил StyleCop).
- Занадто строгі обмеження (наприклад, обов'язкова XML-документація для внутрішніх методів) пом'якшені або вимкнені, щоб не гальмувати швидкість розробки.
- Критичні порушення стилю налаштовані як помилки збірки (Build Errors) або попередження (Warnings), що гарантує чистоту кодової бази і зводить до мінімуму суперечки щодо форматування під час Code Review.

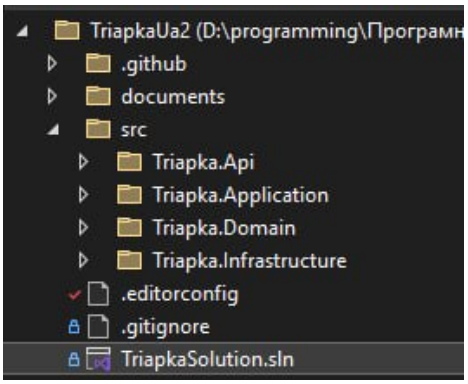
4. Логування та моніторинг: Serilog + Seq

Стандартний логер .NET було замінено на Serilog для реалізації структурованого логування, а також розгорнуто локальний сервер Seq.

- Serilog: Дозволяє записувати логи не просто як плоский текст, а як структуровані JSON-об'єкти з багатим контекстом (властивостями), що критично важливо для подальшого аналізу.
- Seq: Локальний сервер забезпечує зручний веб-інтерфейс для агрегації логів. Він дозволяє виконувати складні SQL-подібні запити по логах, фільтрувати їх за рівнем (Error, Warning, Info) та відстежувати маршрутизацію подій у реальному часі. Маршрутизацію налаштовано паралельно: логи виводяться в консоль (для швидкого дебагу) та відправляються в Seq (для глибокого аналізу).

5. Підтверджувальні матеріали (Скріншоти)

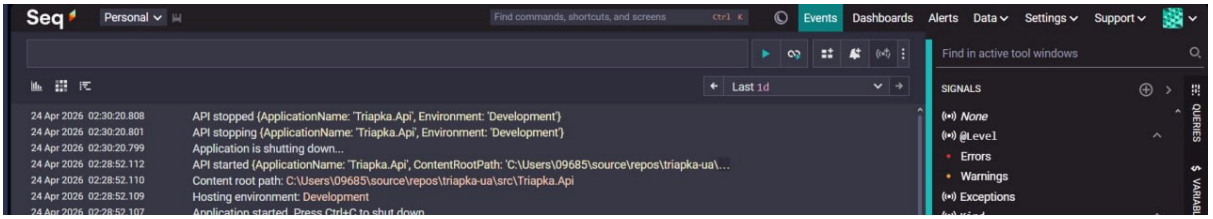
5.1. Структура Solution згідно з Clean Architecture



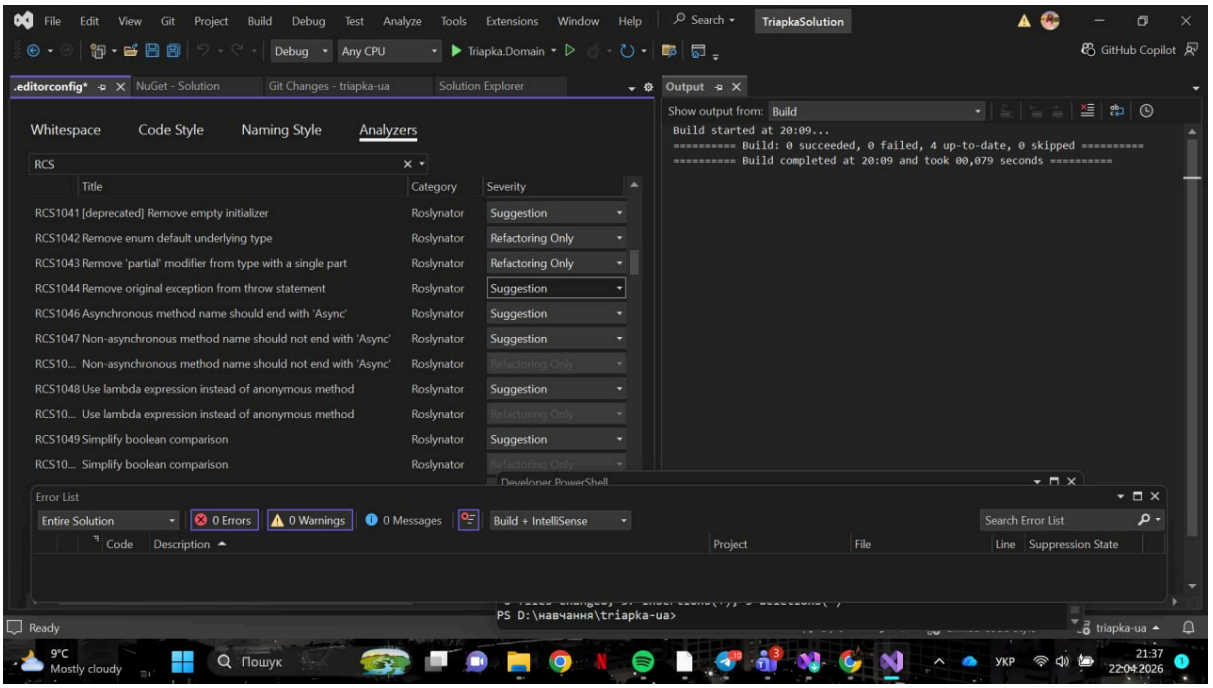
5.2. Успішне розгортання бази даних PostgreSQL

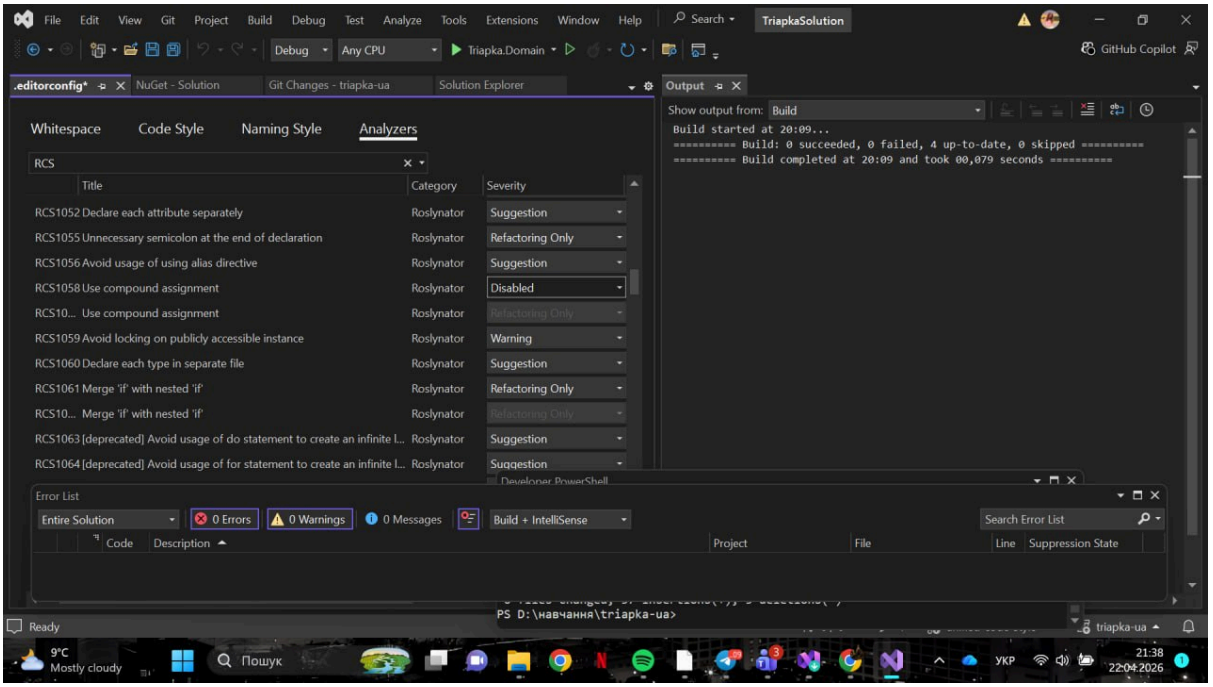
	MigrationId [PK] character varying (150)	ProductVersion character varying (32)
1	20260421205822_InitialCreate	9.0.2

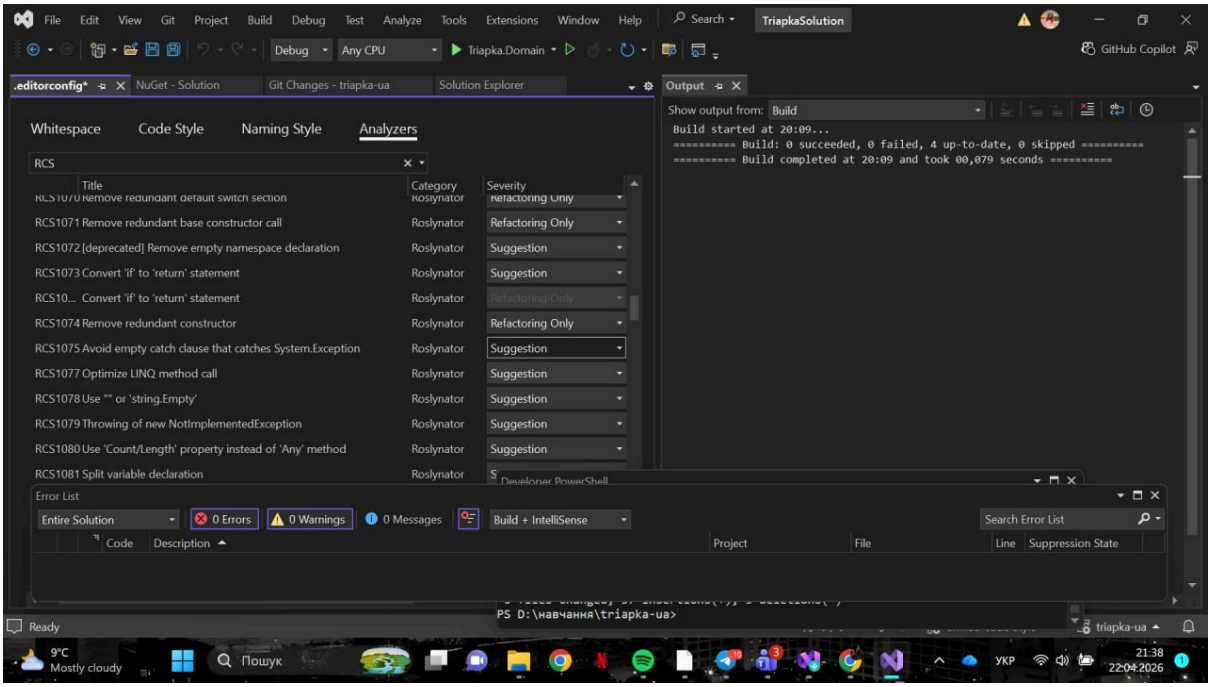
5.3. Структуроване логування у Seq

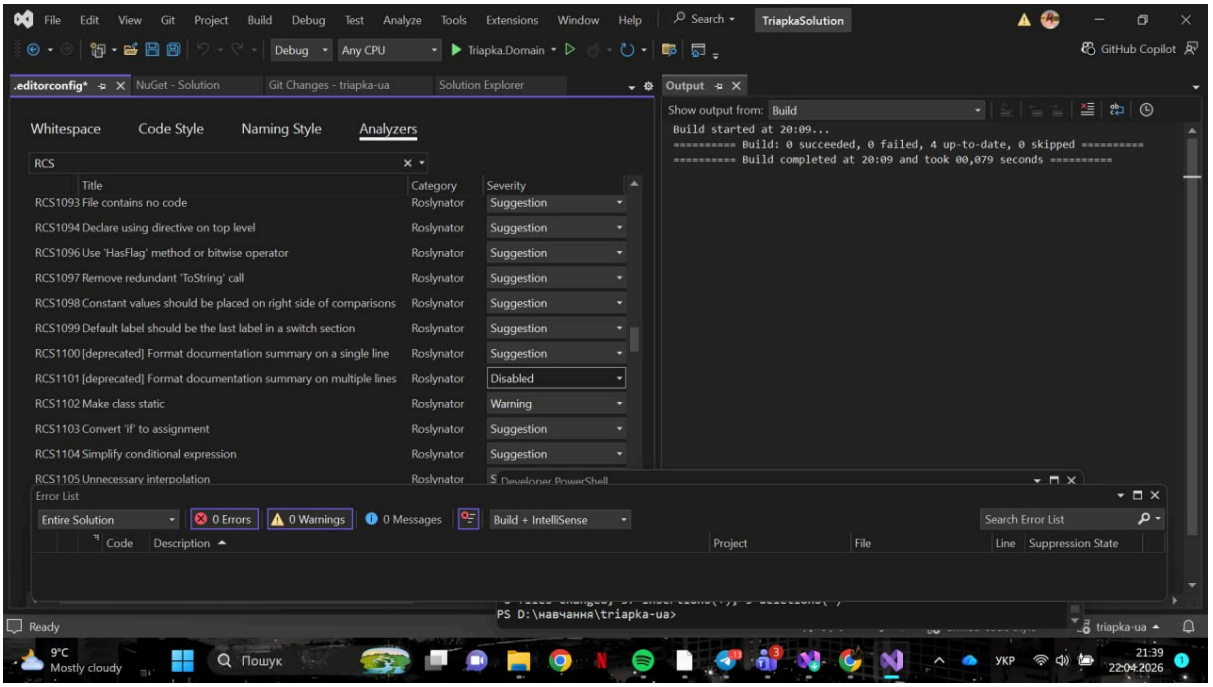


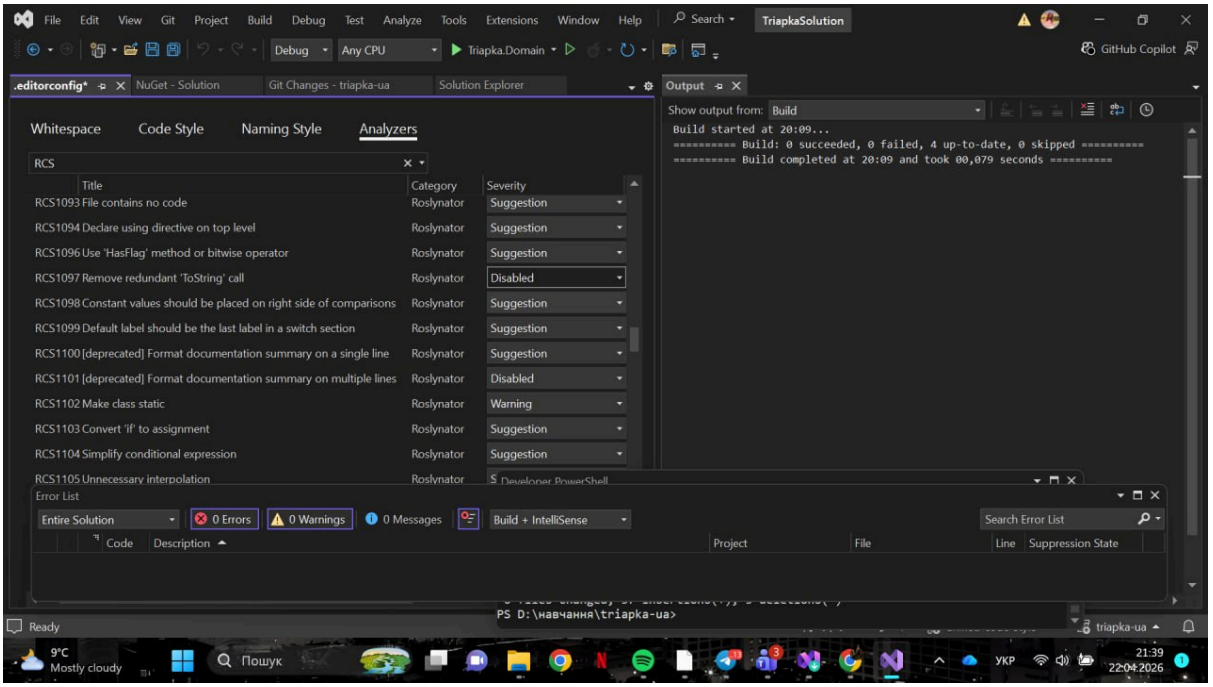
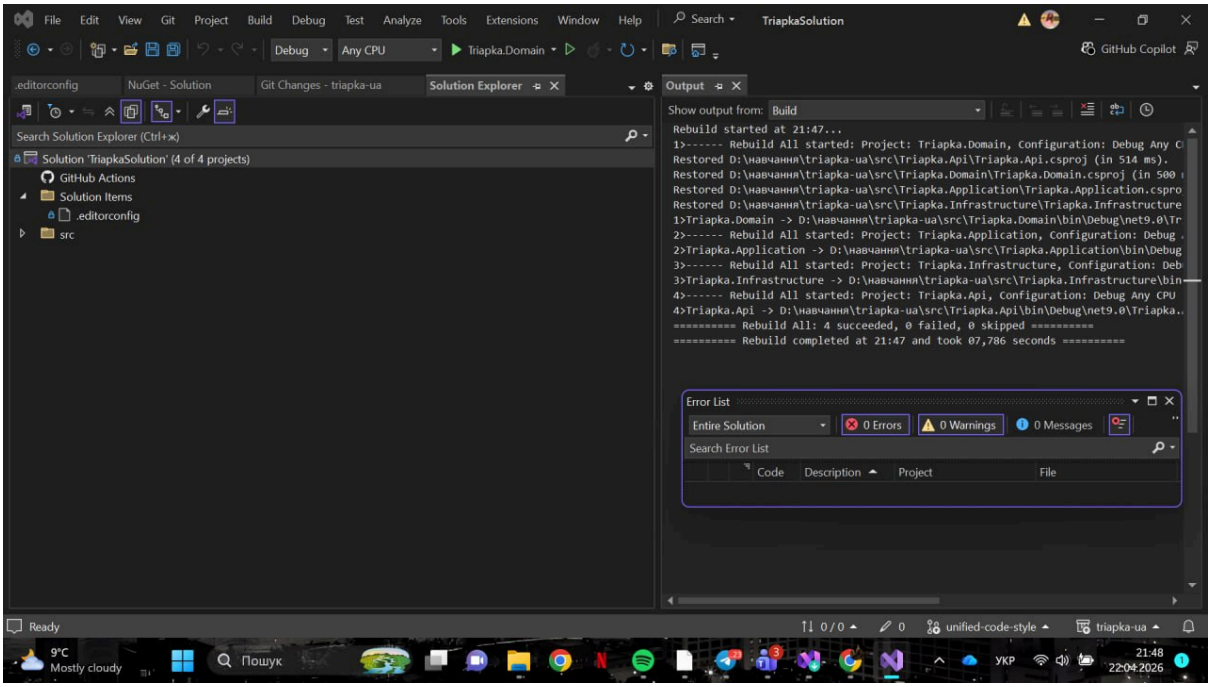
5.4 Правила, які пом’якшені або вимкнені






File Edit View Git Project Build Debug Test Analyze Tools Extensions Window Help
Search
TriapkaSolution
Debug
Any CPU
Triapka.Domain
GitHub Copilot
.editorconfig*
NuGet - Solution
Git Changes - triapka-ua
Solution Explorer
Output
Whitespace
Code Style
Naming Style
Analyzers
RCS
Title
Category
Severity
RCS1070 Remove redundant default switch section
RCS1071 Remove redundant base constructor call
RCS1072 [deprecated] Remove empty namespace declaration
RCS1073 Convert 'if' to 'return' statement
RCS10... Convert 'if' to 'return' statement
RCS1074 Remove redundant constructor
RCS1075 Avoid empty catch clause that catches System.Exception
RCS1077 Optimize LINQ method call
RCS1078 Use "" or 'string.Empty'
RCS1079 Throwing of new NotImplementedException
RCS1080 Use 'Count/Length' property instead of 'Any' method
RCS1081 Split variable declaration
Roslynator
Refactoring Only
Suggestion
Show output from: Build
Build started at 20:09...
========== Build: 0 succeeded, 0 failed, 4 up-to-date, 0 skipped ==========
========== Build completed at 20:09 and took 00,079 seconds ==========
Error List
Entire Solution
0 Errors
0 Warnings
0 Messages
Build + IntelliSense
Search Error List
Code
Description
Project
File
Line
Suppression State
PS D:\навчання\triapka-ua>
Ready
triapka-ua
9°C
Mostly cloudy
Пошук
УКР
21:38
22-04-2026


File Edit View Git Project Build Debug Test Analyze Tools Extensions Window Help
Search
TriapkaSolution
Debug
Any CPU
Triapka.Domain
GitHub Copilot
.editorconfig*
NuGet - Solution
Git Changes - triapka-ua
Solution Explorer
Output
Whitespace
Code Style
Naming Style
Analyzers
RCS
Title
Category
Severity
RCS1093 File contains no code
RCS1094 Declare using directive on top level
RCS1096 Use 'HasFlag' method or bitwise operator
RCS1097 Remove redundant 'ToString' call
RCS1098 Constant values should be placed on right side of comparisons
RCS1099 Default label should be the last label in a switch section
RCS1100 [deprecated] Format documentation summary on a single line
RCS1101 [deprecated] Format documentation summary on multiple lines
RCS1102 Make class static
RCS1103 Convert 'if' to assignment
RCS1104 Simplify conditional expression
RCS1105 Unnecessary interpolation
Roslynator
Suggestion
Disabled
Warning
Show output from: Build
Build started at 20:09...
========== Build: 0 succeeded, 0 failed, 4 up-to-date, 0 skipped ==========
========== Build completed at 20:09 and took 00,079 seconds ==========
Error List
Entire Solution
0 Errors
0 Warnings
0 Messages
Build + IntelliSense
Search Error List
Code
Description
Project
File
Line
Suppression State
PS D:\навчання\triapka-ua>
Ready
triapka-ua
9°C
Mostly cloudy
Пошук
УКР
21:39
22-04-2026



5.5 Підтвердження, що немає помилок після побудови проєкту